

This course has already ended.

« Round 19 (https://plus.cs.aalto.fi/studio_2/k2022/w19/)

Chapter 19.2: Own graphical user interfaces » (<https://plus....>)

CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 19 (https://plus.cs.aalto.fi/studio_2/k2022/w19/)

/ Chapter 19.1: Design patterns

Chapter 19.1: Design patterns

About this page:

Key questions What are Design Patterns?

Topics We get acquainted with design patterns and see how they can be used or how they are part of the Scala language, or its API implementations.

What Will I Do? Mainly read, but also respond to multiple choice questions.

Indicative difficulty level: Easy

Rough Estimate of Workload:? 2 hours

Point Value: 50 (multiple choice questions)

Related Projects: None

Design pattern

Design Pattern is a general description of a working solution (in this context one related to program structures) for common problem. Design patterns can be applied freely. They can also be combined and this is often carried out. This chapter looks at some common problems in object-oriented programming and structures that can potentially solve them.

As an analogy, you can think about different types of wood joints, dance moves, etc. The design pattern describes the general solution and the application area ultimately determines how the pattern is used at any given time.

Why should I know about these?

- Glossary - it facilitates discussion of solutions.
- It makes it easier to understand someone else's program code.
- It possibly resolves a matter related to your own plan.
- At least some of the design models make it easier to recycle and maintain program code because they reduce dependencies between different components.

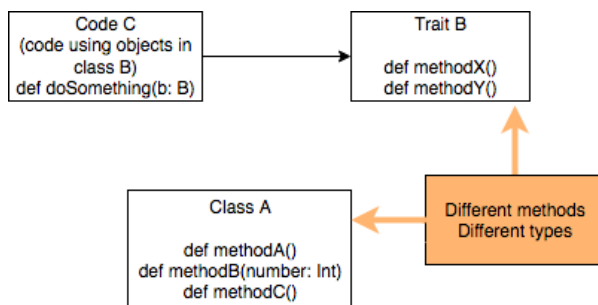
On the other hand, it is also worth noting that many of the models may add a bit to the complexity of the program. Thus, think about whether the model is useful for your own program.

Let's look at some of the commonly used patterns and how they might be used in Scala API, either at the language level or how they could be implemented.

Adapter design pattern

That is, how to make a round piece fit into a square hole

Problem: Some of the existing code works with certain types of objects (such as those that conform a certain trait). However, the type of available objects is not correct. Moreover, neither the class implementing the objects nor code with which it should be used can be changed. For example, the codes may be part of different libraries.

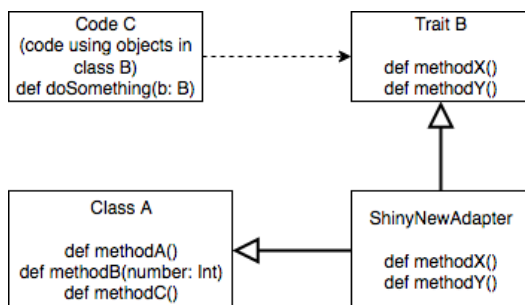


The idea of an adapter design pattern is to adapt the interface of objects (A) to the form (B) that some of the code (C) expects. It also acts as an "interpreter" between these interfaces. When C calls the methods of the interface B, the adapter calls the A methods and transforms the return values into the appropriate ones.

Adapter based on inheritance (class adapter)

If the class of objects (A) can be inherited, and our own code creates all the objects to be used, the matter can be solved by using inheritance. In practice, a new class is created which inherits the original class (A) of the objects and implements the desired interface (B).

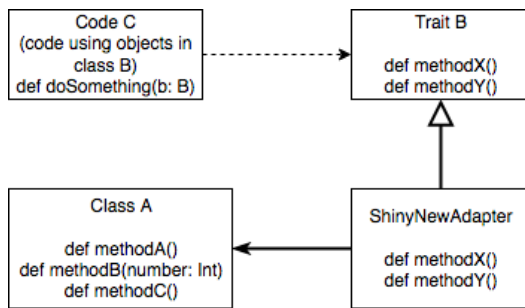
In the new class, the methods of interface B are also implemented so that they call the existing class A methods. When C uses a new class of objects, it uses the original class A methods. The circular block A is obtained in a square hole B.



Adapter based on delegation (object adapter)

If the class of objects (A) cannot be inherited, a new class implementing the interface (B) can be made, whose object has a reference to the object of class (A). When the method of our new class is called, it calls the corresponding methods of the class (A) and returns their return values in a

suitable form.



Example

The SortedSet object maintains the contained objects in order and can only contain entities that match the Ordered interface. We would like to be organize PostPackage class objects by weight, but the class does not meet the Ordered interface, and the PostPackage class cannot be modified because it is part of a finished library.

We fix problem with adapter class (PostPackage class is not described in detail)

```
class PostPackage(val weight: Int)

class OrderedPackageAdapter(p: PostPackage) extends Ordered[PostPackage] {

  def compare(other: PostPackage) = {
    p.weight - other.weight
  }
}
```

Adapter in Scala

In Scala, the programming language level supports automatic adapters and they are widely used, for instance, for automatic upgrading of Java objects with additional methods expected by Scala libraries. For example, String objects get additional features through the StringOps adapter.

```
final class StringOps(override val repr: String) extends AnyVal with StringLike[String] .....
```

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 1

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 5 / 5

Is StringOps a case of an object or class adapter? **Note: Only one response**

☒ Object adapter

☐ Class adapter

✓ Correct!

Question 2 5 / 5

What is the design pattern presented in Chapter 17 that is used in StringLike[String]?

Typeclass

✓ Correct!

Submit

In practice, implementing this adapter is done by means of a so-called implicit transformation. The compiler tries to find and apply the available adapters automatically, if method calls are made for an object such that it does not have. The word `implicit` before the class definition informs the compiler that this is an adapter in which the constructor parameter object is wrapped, if some method is called that is found in the adapter but not in the original class.

Previously seen example with the implicit adapter:

```
import scala.collection.mutable.Buffer

object AdapterExample extends App {

  case class PostPackage(val weight: Int) {
    override def toString = s"Package ($weight kg)"
  }

  implicit class OrderedPackageAdapter(p: PostPackage) extends Ordered[PostPackage] {
    def compare(other: PostPackage) = {
      p.weight - other.weight
    }
  }

  val packets = Buffer[PostPackage](new PostPackage(5), new PostPackage(7), new PostPackage(6))

  packets.sorted foreach println
}
```

Factory Method design pattern

I.e., constructor 2.0 (also know with name *Factory Method pattern*)

A factory method is a method used to create an object instead of a constructor.

- Typically, the method is an apply method in a companion or single object in Scala.
- In practice, the method works like a constructor, returning a reference to an object of the desired type.
- Often it is implemented by calling a constructor within the method.
- Because the constructor is not called directly, some more things can be done:

- For example, make decisions and return an existing object of the desired type, instead of creating a new one.
- Some other type of object can be returned, such that matches the same interface.
- Actions can be carried out on the object before it is returned.

The Factory pattern is used, for example,:

- when the class calling the method itself cannot know which sub-class object it wants to create,
- when you want to parameterize the objects that you create, regardless of the calling class,
 - For example, setting the desired color and text for the user interface buttons automatically. You only need to make the settings once.

- when objects are to be counted, unused objects are recycled, etc.
 - Recycling - Constructor cannot provide a previously created object, but this is possible through the factory method.

```
object FactoryMethod extends App {

  trait GreetingsPrinter {
    def greet(): Unit
  }

  object GreetingsPrinter {

    /**
     * Our factory method. When creating GreetingsPrinter, the factory method selects the right subclass instance and returns it.
     */

    def apply(): GreetingsPrinter = {

      val linux    = """"^Linux"""".r
      val windows  = """"^Windows"""".r
      val mac      = """"^Mac"""".r

      System.getProperty("os.name") match {
        case linux(_)    => LinuxGreeter
        case windows(_) => WindowsGreeter
        case mac(_)      => OSXGreeter
        case _           => throw new Exception("Unknown system")
      }
    }
  }

  /**
   * Alternative results to be returned by the factory method.
   *
   * These could just as well be classes and the factory method would create the objects, if needed.
   */

  object WindowsGreeter extends GreetingsPrinter {
    def greet() = {
      println("Greet (somehow using Windows resources)")
    }
  }

  object OSXGreeter extends GreetingsPrinter {
    def greet() = {
      println("Greet (somehow using Mac resources)")
    }
  }

  object LinuxGreeter extends GreetingsPrinter {
    def greet() = {
      println("Greet (somehow using Linux resources)")
    }
  }

  /**
   * Usage example
   * Note that this prints different text in different operating systems.
   * The factory method within Greetingsprinter provides us an
   * appropriate object from the different alternatives which inherit GreetingsPrinter.
   */

  val printer = GreetingsPrinter()
  printer.greet()
}
```

Regular expressions

Within the apply method, the operating system was identified (mainly in the demo sense) using regular expressions  (http://en.wikipedia.org/wiki/Regular_expression), (Regular Expression), regexp. They can be used to identify certain kind of strings. Regular expressions, finite automata, etc. will certainly be discussed on later courses, but because of their usefulness we will briefly discuss them here.

In Scala, the easiest way to create a regular expression is to write a string limited by triple quotation marks and call it the method `r`. This returns the class `scala.util.matching.Regex`  (<https://www.scala-lang.org/api/current/scala/util/matching/Regex.html>) object. The class documentation contains some good examples of its use with different Scala structures. To find out What kind of expressions you can use, see the corresponding Java documentation in Pattern  (<http://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html>).

Compiling objects which recognize regular expressions is clearly heavier than regular string comparisons. Therefore, it is normally a good idea to use it only when expressions are more complex than our example of operating systems.

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 2

This course has been archived (Thursday, 1 June 2023, 12:00).

Let's try the phrases in practice. The following tasks can be solved with little testing in the REPL. However, those who are more interested, may want to look more closely at, for example, Wikipedia and the documentation mentioned earlier.

Question 1 5 / 5

Create the following regular expression in the REPL `val linux = ""^Linu[sx](es)?""`.r and tell which of the following statements are true.

- ☒ `linux.findFirstIn("Windows") == None`
- ☒ `linux.findFirstIn("Linux") == Some("Linux")`
- ☐ `linux.findFirstIn("linux") == Some("linux")`
- ☒ `linux.findFirstIn("Linus Torvalds") == Some("Linus")`
- ☐ `linux.findFirstIn("Torvalds, Linus") == Some("Linus")`
- ☐ `linux.findFirstIn("Linuxs") == Some("Linuxs")`
- ☒ `linux.findFirstIn("Linuses") == Some("Linuses")`
- ☒ `linux.findFirstIn("Linuxesssss") == Some("Linuxes")`

✓ Correct!

Question 2 5 / 5

Try it in REPL and choose which of the following will recognize only the standard three-letter and three-digit Finnish registration numbers if they appear somewhere in the string. The expression MUST recognize register numbers written in capital letters, but it CAN also recognize lower case register numbers.

- ☐ `[KDRTL]?[a-f][1-8]`
- ☒ `[A-Z]{3}-\d{3}`
- ☐ `$$$-###`
- ☐ `\w\w\w-\d\d\d`
- ☒ `\p{Alpha}{3}-\p{Digit}{3}`

✓ Correct!

Submit

Strategy design pattern

That is, more general algorithms

Suppose there are a number of different ways to solve a computational problem.

In the Strategy pattern, we first write a general implementation of the solution, where alternative aspects are "outsourced" to other classes. Alternate items are written with a trait, through which the general implementation uses the outsourced parts. Now, we can implement various ways of solving the problem, which implement the trait and thus provide a part of the used algorithm. Thus, the entire code does not have to be written for all possible interesting cases.

When the code is written in this way, the solution can be easily changed if desired, for example:

- when text splitting in rows varies by language, or
- comparing items with each other, when sorting them, requires a specific comparison method.

A good example of Scala is the `Ordering[T]` class, which can be used to build different methods to sort items with respect to different criteria. In the example below, the names are first printed alphabetically, then arranged by the second letter of the name. The sorting algorithm itself remains unchanged.

```
object StrategyExample extends App {  
  
  val names = Array("Pekka", "Jaana", "Mika")  
  
  object StrangeOrdering extends Ordering[String] {  
    def compare(a: String, b: String):Int = {  
      a(1) - b(1)  
    }  
  }  
  
  val alphabetical = names.sorted  
  alphabetical.foreach(println)  
  
  println("---")  
  
  val secondLetter = names.sorted(StrangeOrdering)  
  secondLetter.foreach(println)  
}
```

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 3

This course has been archived (Thursday, 1 June 2023, 12:00).

Question 1 5 / 5

Explore the class `scala.math.Ordering`. Click on the "known subclasses" link found at the beginning of its documentation page, and then select the `Ordering` implementation for sorting strings. What is the name of it?

StringOrdering

✓ Correct!

Question 2 5 / 5

Continue to study the documentation of the object you found. Which method you can use for getting reverse alphabetical order?

reverse

✓ Correct!

Submit

State design pattern

That is, program activity in different spaces without a huge amount of ifs

The problem to be solved is that some part of the program works in different situations in substantially different ways. For example, a server in error mode/operational mode, an elevator in the top or bottom floor vs. normal, an adventure game hero in normal mode, in hunger or under the influence of confusion spell. This can be solved with a number of if-sentences or match-ups that check the status and work in different ways in different situations.

However, the idea of the State pattern is to isolate things that depend on the state to separate methods and to change the whole object at once when the state changes. State pattern is closely related to Strategy. When in Strategy one general solution principle was followed, the State pattern uses several ones and can vary between them as needed.

```

object State extends App {

class Elevator(private val MAX:Int = 5) {
  private var floor = 0
  private var state:ElevatorState = BottomFloor

  // The methods work in a different way depending on in which floor the lift is.
  def up()    = state.up()
  def down()  = state.down()
  def break() = state.break()

  def sevenUp() = for (i <- 1 to 7) up()

  //---

  override def toString() = s"Elevator is at state [$state] and located at floor $floor"

  //---

sealed trait ElevatorState {
  def up()    = {
    floor += 1
    if (floor == MAX) state = TopFloor
  }

  def down()  = {
    floor -= 1
    if (floor == 0) state = BottomFloor
  }

  def break() = state = Broken

  val canGoUp:   Boolean = true
  val canGoDown: Boolean = true
  val broken:    Boolean = false
}

case object BottomFloor extends ElevatorState {
  override def down() = {}
  override def up() = {
    super.up()
    state = Normal
  }
  override val canGoDown = false
}

case object TopFloor extends ElevatorState {
  override def up() = {}
  override def down() = {
    super.down()
    state = Normal
  }
  override val canGoUp = false
}

case object Normal extends ElevatorState

case object Broken extends ElevatorState {
  override def down() = {}
  override def up() = {}
  override val canGoDown = false
  override val canGoUp = false
  override val broken = true
}

}

// Example

val e = new Elevator()
println(e)
println("\n>> Up two floors")

```

```

    println("Up two floors")
    e.up()
    e.up()
    println(e)
    println(">> Down three floors")
    e.down()
    e.down()
    e.down()
    println(e)
    println(">> Break it!")
    e.break()
    println(e)
    println(">> It shouldn't move, even if requested")
    e.up()
    println(e)
}

```

Template Method design pattern

This is almost the same as Strategy pattern. The difference is that when the action in Strategy was implemented in a separate object, in the Template Method pattern the "outsourced" methods are written as abstract methods. The class can act in different ways by building different sub-classes in which these methods are implemented.

Iterator design pattern

How can I traverse data structures without knowing how they are internally constructed?

Often, all the elements of a data structure should be traversed (for example, to save them into a file). However, the information can be stored in a variety of structures: trees, lists, graphs, etc., which complicates the code and forces you to study the specialties of each structure. The idea of the Iterator pattern is to allow the structure to be traversed without the need for the user to know the structure at all.

`scala.collection.Iterator[T]` is a textbook example of this pattern. A simplified version of its code is below:

```

trait Iterator[T] {

    /** Test if this iterator has more items to process
     *
     * @return `true` if `next` could return an item, otherwise `false`.
     */
    def hasNext: Boolean

    /** Returns the next value of the iterator.
     *
     * @return next item in this iterator, if `hasnext` returns `true`,
     *         in other cases the action can vary.
     */
    def next(): T
}

```

Thus, there are only two methods to be implemented, one of which is to determine whether more items are available and the other to read the item if there are still some left.

In the example below, the `StudentUnion` register is made to implement `Iterable` feature, that is, one that can be requested an `Iterator` object with the method `iterator`. The last actual line of code shows that `StudentUnion` objects can now be used in the `for` loop as any collection.

```
import scala.collection.mutable.Buffer

object IteratorExample extends App {

  case class Student(name: String)

  // Iterable feature requirement is that class must have
  // a method iterator, through which this kind of structure can be traversed

  class StudentUnion(leader: Student) extends Iterable[Student] {

    def leaderIterator = new Iterator[Student]() {
      var notRead = true // has the leader been read

      def hasNext = notRead

      def next() = {
        notRead = false
        leader
      }
    }

    def iterator = leaderIterator ++ students.iterator // Join the iterators together

    private val students:Buffer[Student] = Buffer[Student]()

    def addMember(person: Student) = {
      students += person
    }
  }

  val union = new StudentUnion(Student("Uolevi"))
  union.addMember(Student("Pekka"))
  union.addMember(Student("Jaana"))
  union.addMember(Student("Mika"))

  // Now when StudentUnion is Iterable, we can traverse it using for loop.
  for (s <- union) println(s)
}
```

Loading assignment...

Command Design Pattern

The idea of the Command pattern is to encapsulate a method call (possibly with parameters) inside an object. This offers many different options.

- By changing command objects, you can vary functionality.
- Objects can - through Command objects - call methods that can be implemented completely apart from them.

The external interface of the commands is defined by a trait or an abstract class. In this case, the executing part of the program can perform them without knowing the contents of the commands beforehand. The Command pattern can be used to implement:

- Undo-redo mechanism
- Functionality of menus in user interfaces
- Recording macros
- and so on...

Example

In the simplest version of our trait, there is a single method that can be called to perform the predefined functionality. In our example, this method is called `execute`.

```
trait Command {  
  def execute()  
}
```

The `DrawLineCommand` is a model example of a class that meets the above interface. A class instance receives the parameters for its predefined task as constructor parameters. However, this is not a requirement of the Command pattern - it is only essential that the object state is preset for the command execution, which occurs only when the `execute` method is performed.

```
case class DrawLineCommand (drawingArea: Picture, x1: Int, y1: Int, x2: Int, y2: Int) extends Command {  
  
  def execute() = {  
    drawingArea.drawLine( x1, y1, x2, y2 )  
  }  
}
```

Finally, we look at how the commands are created and executed. Here's a script that draws a filled square. Note `FillCommand` among the line draw commands, and that it doesn't change the for loop where commands are executed. In our example, the values were fixed, but in reality the values for the command could be get from the user's mouse clicks. In this way, we could be built even a macro that can draw squares.

```
/* code */  
  
doTheseLater.add( DrawLineCommand(thisPic, 0, 0, 15, 0) )  
doTheseLater.add( DrawLineCommand(thisPic, 15, 0, 15, 15) )  
doTheseLater.add( DrawLineCommand(thisPic, 15, 15, 0, 15) )  
doTheseLater.add( DrawLineCommand(thisPic, 0, 15, 0, 0) )  
doTheseLater.add( FillCommand(thisPic, 10, 10, color.RED) )  
  
/* code */  
  
for (thisCommand <- doTheseLater){  
  thisCommand.execute()  
}  
  
/* code */
```

Command pattern and Undo mechanism

The command pattern can also be used to implement the so-called Undo-redo-mechanism. Its implementation is based on a solution in which, instead of one execute method, two "execute methods" are built for the command interface: Undo and Redo.

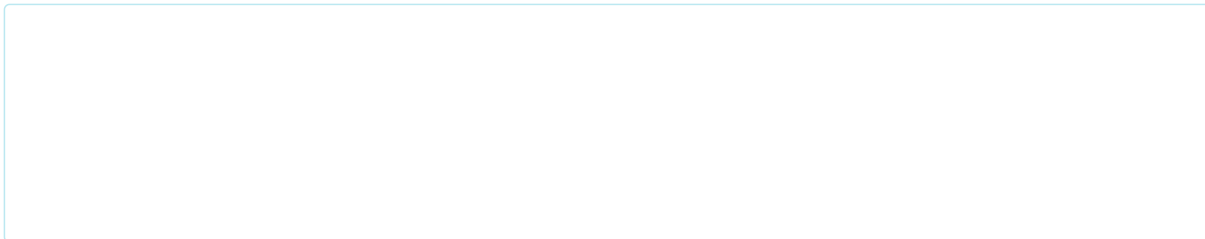
In practice, Redo is essentially the same as the execute method of a simple Command-pattern. Usually, some more information is required for Undo. You are likely to guess how the code below works.

```
case class RenameCommand(person: Person, newName: String, oldName: String) extends Command {  
  
    def undo(): Unit = {  
        person.setName(oldName)  
    }  
  
    def redo(): Unit = {  
        person.setName(newName)  
    }  
}
```

The last part of the mechanism is the handling of commands. There are two *stacks* for the commands. A stack is a data structure that works in the same way as real-life stacks (e.g., stack of plates in a lunch restaurant): you do not touch items in the middle of the stack, but you can place a new item on top of the stack and remove the item from the top of the stack.

Whenever a command is executed, it is put in an undo stack for the future. If Undo is performed, take the top command of the undo stack, execute its undo method and move the command to redo stack. The Redo operation works the opposite.

Note: If someone executes a new command when there are commands in the redo stack, they must be discarded.

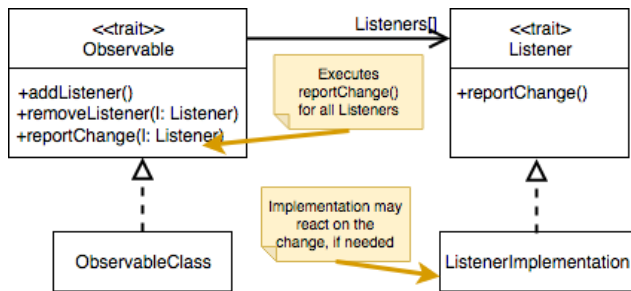


Observer design pattern

The Observer design pattern allows to build an action where a change in one entity causes "automatically" a change in all dependent objects. In practice, an *interesting object* (or Observable object) has a list on which *interested* (Listener, Observer) objects can enroll. By enrolling, they ask the interesting object to tell about its changes. This pattern is widely used in building graphical user interfaces.

When something happens, then every object in the list is traversed and its "notification method" is called. Usually, the method is also given a little message (*Event*) that describes what happened. One essential feature of the pattern is the thin dependence of Observable object on the Listeners. The first one needs not and should not know about the Listener anything except that it meets the "listener" interface.

In the chapter 18.3__, this pattern is used to build an animation by listening to a regular recurring event.

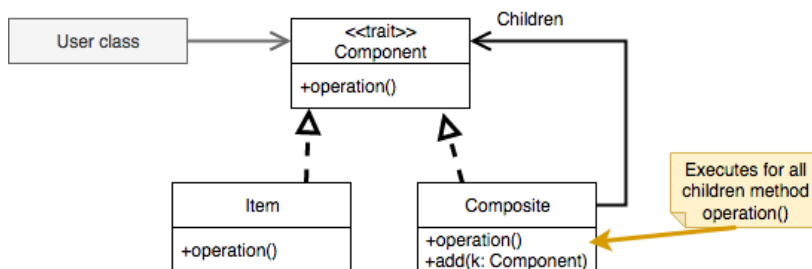


Composite design pattern

In some data structures (e.g., trees), the structure nodes consist of smaller nodes (children), which in turn are composed of smaller nodes, and so on.

Composite is another design pattern that often comes up with graphical user interfaces. Its most important idea is that for a complex group of objects, an operation can be performed in the same way as for a single object.

- Drawing a window on the screen also draws the panels it contains, the buttons it contains, and so on.
- The number of words in a text could be calculated by asking the document for the number, and it asks the same from chapters, and chapters from paragraphs.
- The DOM tree of an HTML document can be used to search for a node recursively.
- and so on



In the example below, we count the number of characters in the book using the countChars method.

```
import scala.collection.mutable.Buffer

object BookExample extends App {

  trait TextComponent {
    def countChars: Int
  }

  case class Sentence(val s: String) extends TextComponent {
    val countChars = s.length
  }

  class Section extends TextComponent {
    val contents = Buffer[Sentence]()

    def apply(text: Sentence*): Section = {
      contents ++= text
      this
    }

    def countChars = contents.map(_.countChars).sum
  }

  class Chapter extends TextComponent {
    val contents = Buffer[Section]()

    def apply(s: Section*) = {
      contents ++= s
      this
    }

    def countChars = contents.map(_.countChars).sum
  }

  val book      = new Chapter
  val introduction = new Section
  val theEnd     = new Section

  /* creative use of apply method. Note how readable the code is.
  */

  book(
    introduction(
      Sentence("Call me Ishmael"),
      Sentence("Some years ago")
    ),
    theEnd(
      Sentence("At day-break, the three mast-heads were punctually manned afresh.")
    )
  )

  println(book.countChars)
}
```

Summary

A few examples of Design Pattern were presented above. These are generic solutions to common problems. There is no fixed and one way to implement a particular pattern, but when you learn to know them, it will be easier study others' code and discuss the structure of the program. Keep in mind that using patterns is not a goal itself, but they should be applied when they are useful. In some programming languages, the need for some design pattern may not even exist; in another the same pattern may be particularly relevant.

The models presented above were all a part of the so-called Gang of Four Design patterns. Only some of them were presented. There are many other common-purpose patterns, and much more application-specific patterns. For example, in Wikipedia you can see one list of different design patterns  (http://en.wikipedia.org/wiki/Software_design_pattern).

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately